CPSC-406 Report

Angelina House
Chapman University

May 17, 2025

Abstract

Contents

1 Introduction

2 Week by Week

2.1 Week 1

Notes

In week one we were introduced to basic automata therory and given multiple examples of how they can be used. Our main example was one of a parking meter where we were introduced to states, transitions, and final states. We also went over naming specifications, word lists and equations, and finite automata.

Homework

Question 1: Characterize all the accepted words (i.e., describe exactly those words that get recognized).

We can first start characterizing the accepted words by looking at the given graph and gaining an understanding of the automaton. In this automaton there we have a coin machine which needs 25 cent and can only take 5 and 10 cent coins. We can see that the machine will accept the word if the sum of the coins is 25 cents. With this we can characterize the accepted words as follows:

5 5 5 5 5
5 5 5 10
5 5 10 5
5 10 5 5
10 5 5 5
10 10 5
10 5 10
5 10 10

Question 2: Characterize all the accepted words. Can you describe them via a regular expression?

We can first start charaterizing the accepted words by understanding the graph, states, and transitions. In this graph, pay moves the turnstile from locked to unlocked, push turns the turnstile from unlocked to locked, if the turnstile is already unlocked a pay occurs then it remains unlocked and if the turnstile is locked and a push is used then nothing happens. With this understanding we can formulate that atleast one pay is needed to unlock the turnstile, any number of pay's can happen while the turnstile is unlocked and a push after a pay locked the turnstile making it need another pay to unlock again. With this we can characterize the accepted words as follows:

```
pay
pay pay
pay pay pay
pay push pay
pay pay push pay
pay pay pay push pay
```

Overall, the pattern follows:

```
pay (push pay)*
```

Reading Summary

There was no readings or questions for this week.

2.2 Week 2

Notes

This week we covered Derterministic Finite Automata (DFA) and continued to build on our definition on automata from week one. We focused on the bridge problem where we had to determine the best way to get across and then turn that into a DFA. We also had our first lab where we focused on word processing and constructing automata.

Homework

Question 1: Determine for the following words, if they are contained in L1, L2, or L3

The equations are:

$$L_1 := \{x01y \mid x, y \in \Sigma^*\}$$
$$L_2 := \{w \mid |w| = 2^n \text{ for some } n \in \mathbb{N}\}$$
$$L_3 := \{w \mid |w|_0 = |w|_1\}$$

Using the given chart and the given equations we can come to the following conclusions.

Word	L_1 (Contains "01")	L_2 (Length 2^n)	L_3 (Equal 0s and 1s)
$w_1 = 10011$	✓	x	x
$w_2 = 100$	x	x	x
$w_3 = 10100100$	✓	✓	×
$w_4 = 1010011100$	✓	×	✓
$w_5 = 11110000$	×	✓	✓

By following the the language equations, conditions, and comparing using the word we can determine whether the word would be accepted in the equation.

Question 2: Let Σ be some fixed alphabet and $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton. The DFA $\mathcal{A}$ can "process" a word by tracing through it via a sequence of transitions corresponding to the letters of the word. Consider the paths corresponding to the words:

$$w_1 = 0010, \quad w_2 = 1101, \quad \text{and} \quad w_3 = 1100.$$

For which of these words does their run end in the accepting state?

w_1 and w_2 end in the accepting state. Following the given graph and going through the word we can determine that w_1 and

Reading Summary and Questions

Section 2.1 introduces finite automata in a straightforward easy to comprehend way by explaining how they work and why they're useful. A finite automaton is a simple computational model that processes input by transitioning between a set number of states. It follows predefined rules to determine what state to move to based on the input it receives. The section gives real world examples where finite automata are applied, such as text searching, lexical analysis in programming languages, digital circuit design, and verifying network protocols. These machines help computers recognize patterns and process sequences efficiently.The section discusses two main types, Deterministic Finite Automata (DFA), where each state has only one possible transition per input, and Nondeterministic Finite Automata (NFA), which allows multiple possible transitions. Both are equally as powerful but NFA's are typically more flexible. This section doesn't get too technical yet it just lays the foundation for the more detailed discussions coming later. The key takeaway is that while finite automata are simple, they play an essential role in many areas of computer science, especially in recognizing and processing data.

My questions this week was regarding the complexities of finite automata. It was "Is there a point where an automata gets too complicated to the point where its not the best option anymore? If so, is there any alternatives to the automaton that might work better in this situation?"

2.3 Week 3

Notes

Homework

Homework two include two lab questions from our week two lab. The labs focus on developing and altering a DFA from code as well as understanding the varuous actions that can be done to them. In the first question, we are asked to implement the run function as well as define the A_1 and A_2 from the imagaes given. In the following question, we are asked to consider a new automaton A_3 and implement a new refuse function. This is all done within three different python files, dfa.py, dfa$_e$x01.*pyanddfa*$_e$x03.*py*.

dfa.py:

```
# dfa.py
class DFA:
    def __init__(self, Q, Sigma, delta, q0, F):
        self.Q = Q # Set of states
        self.Sigma = Sigma # Alphabet
        self.delta = delta # Transition function
        self.q0 = q0 # Initial state
```

```
        self.F = F # Accepting states

    def __repr__(self):
        return f"DFA({self.Q},\n\t{self.Sigma},\n\t{self.delta},\n\t{self.q0},\n\t{self.F})"

    def run(self, w):
        current_state = self.q0
        for symbol in w:
            if symbol not in self.Sigma:
                return False
            current_state = self.delta.get((current_state, symbol))
            if current_state is None:
                return False
        return current_state in self.F # Accept if in accepting state

    def refuse(self):
        # new dfa that refuses input
        return DFA(self.Q, self.Sigma, self.delta, self.q0, set())

# Example DFA
if __name__ == "__main__":
    Q = {'q0', 'q1'}
    Sigma = {'0', '1'}
    delta = {
        ('q0', '0'): 'q0',
        ('q0', '1'): 'q1',
        ('q1', '0'): 'q1',
        ('q1', '1'): 'q0'
    }
    q0 = 'q0' # Initial state
    F = {'q1'} # Accepting state

    dfa = DFA(Q, Sigma, delta, q0, F)
    print(dfa.run('101')) # false
    print(dfa.run('100')) # true
```

***** dfa_ex01.py: *****

```
from dfa import DFA

def generate_words():
    words = [""]
    alphabet = ['a', 'b']

    # Generate words
    for first in alphabet:
        words.append(first)
        for second in alphabet:
            words.append(first + second)
            for third in alphabet:
                words.append(first + second + third)

    return words

def __main__():
    # Define DFA A1
```

```
    A1 = DFA(
        Q={1, 2, 3, 4},
        Sigma={'a', 'b'},
        delta={
            (1, 'a'): 2, (1, 'b'): 4,
            (2, 'a'): 2, (2, 'b'): 3,
            (3, 'a'): 3, (3, 'b'): 3,
            (4, 'a'): 4, (4, 'b'): 4
        },
        q0=1,
        F={3}
    )

    # Define DFA A2
    A2 = DFA(
        Q={1, 2, 3},
        Sigma={'a', 'b'},
        delta={
            (1, 'a'): 2, (1, 'b'): 1,
            (2, 'a'): 3, (2, 'b'): 1,
            (3, 'a'): 3, (3, 'b'): 3
        },
        q0=1,
        F={3}
    )

    words = generate_words()
    automata = [A1, A2]

    # Test words on automata
    for X in automata:
        # print(f"{X.__repr__()}") # prints DFA representation
        for w in words:
            print(f"{w}: {X.run(w)}") # prints true or false
        print("\n")

__main__()
```

***** dfa_ex03.py: *****

```
from dfa import DFA

def generate_words():
    words = [""]
    alphabet = ['a', 'b']

    for first in alphabet:
        words.append(first)
        for second in alphabet:
            words.append(first + second)
            for third in alphabet:
                words.append(first + second + third)

    return words

def __main__():
```

```
    # Define DFA A
    A = DFA(
        Q={1, 2, 3, 4},
        Sigma={'a', 'b'},
        delta={
            (1, 'a'): 2, (1, 'b'): 4,
            (2, 'a'): 3, (2, 'b'): 4,
            (3, 'a'): 3, (3, 'b'): 3,
            (4, 'a'): 2, (4, 'b'): 4
        },
        q0=1, # Initial state
        F={3, 4} # Accepting states
    )

    # Define DFA A0 (refuse)
    A0 = A.refuse()

    words = generate_words()
    automata = [A, A0]

    # Test words on automata
    for X in automata:
        # print(f"{X.__repr__()}") # Prints DFA representation
        for w in words:
            print(f"{w}: {X.run(w)}") # Prints True or False
        print("\n")

__main__()
```

Reading Summary and Questions

Section 2.2 focuses on Deterministic Finite Automata (DFA), which are computational models used to process input strings and determine whether they belong to a specific language. A DFA consists of a finite set of states Q, a defined input alphabet Σ, transition rules δ that dictate how the automaton moves between states, a starting state q_0, and one or more accepting states F that signify valid inputs. The operation of a DFA is systematic: it begins in the initial state and processes an input string one symbol at a time, following a unique transition for each symbol. Once the entire input has been read, the DFA determines acceptance based on its final state. If it ends in an accepting state, the input is considered valid; otherwise, it is rejected. Formally, a DFA is defined as a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is the set of states, Σ is the input alphabet, δ is the transition function that maps states and inputs to new states, q_0 is the initial state, and F is the set of accepting states. DFAs are widely used in applications such as text searching, lexical analysis in programming, and pattern recognition. Their deterministic nature makes sure that for any given input, the transition path is unique making them both predictable and efficient for structured data processing.

This week I was curious about the limits of a DFA. My question was "The book emphasized that DFA's can only process one string at a time but would we ever get to the point where it would be able to process multiple strings at one time? Would it make the DFA simpler or more complex and at what point would we have to get quantum computing involved?"

2.4 Week 4

Notes

Homework

Question 1: Describe the language accepted by $\mathcal{A}^{(2)}$ The language accepted by $\mathcal{A}^{(2)}$ is that words must have an odd length and every odd indexed position must be a letter 'a'. Some accepted words are: $a, aba, aba, abaaaa, aaaaaaaa$

Compute the extended transition functions:

1. $\widehat{\delta^{(1)}}(1, abaa)$ for $\mathcal{A}^{(1)}$ Following the transitions on the input $abaa$:

$$\begin{aligned} &\text{Start at state } q_1. \\ &\text{Read } 'a' \quad \Rightarrow \quad q_2. \\ &\text{Read } 'b' \quad \Rightarrow \quad q_3. \\ &\text{Read } 'a' \quad \Rightarrow \quad q_4. \\ &\text{Read } 'a' \quad \Rightarrow \quad q_5. \\ &\text{Making the final state} \quad q_5 \end{aligned}$$

2. $\widehat{\delta^{(2)}}(1, abba)$ for $\mathcal{A}^{(2)}$ Following the transitions on the input $abba$:

$$\begin{aligned} &\text{Start at state } p_1. \\ &\text{Read } 'a' \quad \Rightarrow \quad p_2. \\ &\text{Read } 'b' \quad \Rightarrow \quad p_3. \\ &\text{Read } 'b' \quad \Rightarrow \quad p_4. \\ &\text{Read } 'a' \quad \Rightarrow \quad p_5. \\ &\text{Making the final state} \quad p_5 \end{aligned}$$

Question 2:
Construct the product automaton $\mathcal{A}$ that recognizes $L(\mathcal{A}^{(1)}) \cap L(\mathcal{A}^{(2)})$.

- **States:** $Q = Q^{(1)} \times Q^{(2)}$ (pairs of states from $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$).
- **Alphabet:** $\Sigma = \{a, b\}$.
- **Transition function:**
$$\delta((q^{(1)}, q^{(2)}), x) = (\delta^{(1)}(q^{(1)}, x), \delta^{(2)}(q^{(2)}, x))$$
- **Initial state:** $q_0 = (q_0^{(1)}, q_0^{(2)})$.
- **Accepting states:** $F = F^{(1)} \times F^{(2)}$ (both states must be accepting).

Language of $\mathcal{A}$:

- Words must have **odd length**.
- Every letter at an **odd position** must be a.

- No two consecutive letters can be the same.

Examples of words in $L(\mathcal{A})$:

- a, aba, $ababa$

Examples of words not in $L(\mathcal{A})$:

- b (does not start with a)
- aa (two consecutive a's)
- $abaa$ (even length)
- $abba$ (incorrect odd-positioned character)

$L(\mathcal{A})$ is regular because regular languages are closed under intersection. Since $L(\mathcal{A}^{(1)})$ and $L(\mathcal{A}^{(2)})$ are regular, their intersection $L(\mathcal{A})$ is also regular.

Reading Summary and Questions

Section 2.3 covers Nondeterministic Finite Automata (NFA) and how they differ from Deterministic Finite Automata (DFA). The key difference is that, unlike a DFA where each state has exactly one transition per input, an NFA allows multiple possible transitions, or none at all, for the same input symbol. This make an NFA capable of following multiple computational paths at once, making it more flexible. A string is accepted if at least one of these paths leads to an accepting state. The section also introduces the extended transition function, which determines all the possible states an NFA can reach after processing a string. NFAs recognize the same class of regular languages as DFAs. Any NFA can be converted into an equivalent DFA using the subset construction method but, this process can significantly increase the number of states, sometimes exponentially. Regardless, NFAs are often more compact and easier to construct, making them useful in applications like pattern matching and lexical analysis. While DFAs and NFAs are equivalent in terms of what they can recognize, NFAs provide a more convenient and flexible approach to defining automata.

My question this week was regarding certain use cases of NFA's and DFA's. What are some examples where a DFA would be prefered over and NFA and vice versa? What are some of the drawbacks to using one or the other in these situations?

2.5 Week 5

Notes

Homework

Question 1:
We can understand A as an NFA redefining the transition functions to allow for possible next states making it compatible with the power set of Q. If this is correctly done then the NFA interpretation from state q on input a will result in a non determinitic transitions to any state in the set. For any given DFA $M = (Q, \Sigma, \delta, q_0, F)$, the corresponding NFA $M' = (Q', \Sigma, \delta', q_0', F')$ is defined as:

- $Q' = Q$
- $\delta'(q, a) = \{\delta(q, a)\} \quad \forall q \in Q, a \in \Sigma$
- $q_0' = q_0$
- $F' = F$

These descriptions ensure that the NFA accepts the same language as the original DFA since each transition remains deterministic. Since $\delta(q, a)$ in a DFA always returns a single state, the corresponding NFA transition function also only produces singleton sets. This means that for every input string $w \in \Sigma^*$, the sequence of transitions followed by the DFA is exactly the same as in the NFA. Because both automata start from the same initial state q_0 and have the same set of accepting states F, the set of accepted strings is identical.

Question 2:
The given NFA A language can be described as

$$L = \{w \in \{a, b\}^* \mid w \text{ contains the substring "ab"}\}$$

Any string in this language must contain at least one occurrence of the substring "ab" and the substring "ab" can appear anywhere in the string—at the beginning, middle, or end. A can be specified as:

$$A = (Q, \Sigma, \delta, q_0, F)$$

where:

- $Q = \{q_0, q_1, q_2\}$ is the set of states.
- $\Sigma = \{a, b\}$ is the input alphabet.
- q_0 is the initial state.
- $F = \{q_0, q_1\}$ is the set of accepting states.
- δ is the transition function defined as follows:

We compute the extended transition function $\hat{\delta}(q_0, 10110)$ by processing the input string one symbol at a time:

1. **Start State:** q_0
2. **Input 1:** From q_0, on input 1, transition to q_1.
$$q_0 \xrightarrow{1} q_1$$
3. **Input 0:** From q_1, on input 0, transition to q_2.
$$q_1 \xrightarrow{0} q_2$$
4. **Input 1:** From q_2, on input 1, transition to q_1.
$$q_2 \xrightarrow{1} q_1$$
5. **Input 1:** From q_1, on input 1, transition to q_1 (self-loop).
$$q_1 \xrightarrow{1} q_1$$
6. **Input 0:** From q_1, on input 0, transition to q_2.
$$q_1 \xrightarrow{0} q_2$$

$$\hat{\delta}(q_0, 10110) = q_2$$

Computing Paths for $v = 1100$

We begin at the start state q_0 and process $v = 1100$ step by step.

1. **Input 1:** From q_0, on input 1, transition to q_1.
$$q_0 \xrightarrow{1} q_1$$
2. **Input 1:** From q_1, on input 1, transition to q_1 (self-loop).
$$q_1 \xrightarrow{1} q_1$$
3. **Input 0:** From q_1, on input 0, transition to q_2.
$$q_1 \xrightarrow{0} q_2$$
4. **Input 0:** From q_2, on input 0, transition to q_3.
$$q_2 \xrightarrow{0} q_3$$

Final State for $v = 1100$: q_3 (accepting state)

Computing Paths for $w = 1010$

We begin at the start state q_0 and process $w = 1010$ step by step.

1. **Input 1:** From q_0, on input 1, transition to q_1.
$$q_0 \xrightarrow{1} q_1$$
2. **Input 0:** From q_1, on input 0, transition to q_2.
$$q_1 \xrightarrow{0} q_2$$
3. **Input 1:** From q_2, on input 1, transition to q_1.
$$q_2 \xrightarrow{1} q_1$$
4. **Input 0:** From q_1, on input 0, transition to q_2.
$$q_1 \xrightarrow{0} q_2$$

Final State for $w = 1010$: q_2 (not an accepting state)

The power set $P(Q)$, which represents the states of the equivalent DFA, can be represented as:

$$P(Q) = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}, \{q_0, q_1\}, \{q_1, q_2\}, \{q_2, q_3\}, \{q_1, q_3\}, \{q_0, q_2\}, \{q_0, q_3\}, \{q_0, q_1, q_2\}, Q\}$$

The DFA $\mathcal{A}^D$ accepts the same language as $\mathcal{A}$. By construction, the power set method guarantees that $\mathcal{A}^D$ accepts exactly the same strings as $\mathcal{A}$. Since every NFA has an equivalent DFA, and $\mathcal{A}^D$ was obtained directly from $\mathcal{A}$ via determinization. After minimization, a smaller DFA may exist that recognizes $L(\mathcal{A})$ with fewer states than the DFA obtained via power set construction. The minimal DFA provides a more efficient representation of the same language.

Reading Summary and Questions

Section 3.1 introduces regular expressions, which provide a structured way to define patterns in strings. They use operations like union ($\cup$) to represent choices, concatenation ($\cdot$) to combine elements, and the Kleene star ($*$) to indicate repetition. These rules help define regular languages and are widely used in search functions, text processing, and programming. Section 3.2.1 explains how to convert a DFA into a regular expression by systematically breaking it down into smaller expressions that describe all possible paths. This method gradually builds a final regular expression that represents the entire language of the DFA. Section 3.2.2 presents another approach called state elimination, where states are removed one at a time, replacing their transitions with equivalent regular expressions until a single expression remains. While this method is straightforward, it can sometimes result in overly complex expressions. In summary, these sections highlight the fundamental equivalence between DFAs and regular expressions, showing that any language recognized by a DFA can be represented by a regular expression, and vice versa.

My question this week was about converting a DFA into a regular expression and what challenges or issues might come up where it makes one approach better than the other?

2.6 Week 6

Notes

Homework

3.2.1:

Given DFA:

	0	1
$\rightarrow q_1$	q_2	q_1
q_2	q_3	q_1
$*q_3$	q_3	q_2

(a) Direct transitions $R_{ij}^{(0)}$:

$$\begin{bmatrix} 1 & 0 & \emptyset \\ 1 & \emptyset & 0 \\ \emptyset & 1 & 0 \end{bmatrix}$$

(b) Allowing paths through q_1, the key result is:

$$R_{13}^{(1)} = \emptyset$$

(c) Allowing paths through q_1 and q_2:

$$R_{13}^{(2)} = 1^*0(1^*0)^*0$$

(d) Regular expression for the language:

$$L = 1^*0(1^*0)^*0$$

(e) After eliminating q_2, the result is the same:

$$L = 1^*0(1^*0)^*0$$

3.2.2:

Given DFA:

	0	1
$\rightarrow q_1$	q_2	q_3
q_2	q_1	q_3
$*q_3$	q_2	q_1

(c) One way to reach q_3 from q_1 is:

$$1, \quad 01, \quad 0101, \quad \ldots$$

A regular expression is:

$$(01)^*1 + 0(10)^*1$$

(d) Regular expression for the language:

Strings that end at q_3, matching:

$$(01)^*1 + 0(10)^*1$$

4.4.1

- (a) To find distinguishable state pairs, we first marked all pairs where one state is accepting (D) and the other is not. Then, we used transitions to mark more pairs based on previously marked ones.
- The equivalent state pairs are: (A, B) and (F, G).
- (b) After grouping equivalent states, we get the following minimized DFA:

State	0	1	Final?
[A, B]	[A, B]	[A, B]	No
[C]	[D]	[A, B]	No
[D]	[D]	[A, B]	Yes
[E]	[D]	[F, G]	No
[F, G]	[F, G]	[F, G]	No
[H]	[F, G]	[D]	No

4.4.2

- (a) We marked all pairs where one state is accepting (C, F, I) and the other is not, and used transitions to find more distinguishable pairs.
- The only equivalent pair is: (G, H).
- (b) The minimized DFA has the following transitions:

State	0	1	Final?
[A]	[B]	[E]	No
[B]	[C]	[F]	No
[C]	[D]	[G, H]	Yes
[D]	[E]	[G, H]	No
[E]	[F]	[I]	No
[F]	[G, H]	[B]	Yes
[G, H]	[I]	[B]	No
[I]	[A]	[E]	Yes

Reading Summary and Questions

Section 4.4 covers how to determine if two deterministic finite automata (DFAs) are equivalent and how to simplify a DFA by minimizing it. Two DFAs are considered equivalent if they accept exactly the same set of strings, meaning they recognize the same language. One way to test this is by creating a new DFA that accepts only the strings where the original two DFAs disagree—if this new DFA accepts nothing, then the two are equivalent. The section also introduces a process for minimizing a DFA by merging states that behave the same for all inputs. This is done using the table-filling algorithm, which helps identify indistinguishable states—those that can't be told apart by any string. Once the indistinguishable states are grouped, a new, smaller DFA is created that still accepts the same language. This minimized DFA is guaranteed to be the smallest possible version for that language and is unique up to state naming. Minimizing DFAs is helpful in real-world applications like text processing, search algorithms, and compiler design because it makes the automaton more efficient. Overall, Section 4.4 provides practical techniques for comparing and simplifying DFAs while ensuring their behavior stays the same.

My question this week was asking if there are any situations where minimizing a NFA would it result in changing the language that is recognized?

2.7 Week 7

Notes

Homework

Turing Machine Code (in 5-tuple format):

```
(q_even, 0, 0, R, q_odd)
(q_even, 1, 1, R, q_even)
(q_even, _, _, R, q_accept)
(q_odd, 0, 0, R, q_even)
(q_odd, 1, 1, R, q_odd)
(q_odd, _, _, R, q_reject)
```

Transition Table:

Current State	Read Symbol	Write Symbol	Move	Next State
`q_even`	0	0	R	`q_odd`
`q_even`	1	1	R	`q_even`
`q_even`	_	_	R	`q_accept`
`q_odd`	0	0	R	`q_even`
`q_odd`	1	1	R	`q_odd`
`q_odd`	_	_	R	`q_reject`

Reading Summary and Questions

Sections 8.1 and 8.2 explore the limits of what computers are capable of. Section 8.1 introduces the idea of undecidable problems—problems that no program can solve for every possible input. One example they use is checking whether a program prints "hello, world." Even though it sounds simple, there's no general algorithm that can do this for all cases without running into logical contradictions. The section also talks about reductions, where you show that one problem is at least as hard as another, which helps prove when something is truly undecidable. Section 8.2 introduces the Turing Machine, a basic but powerful model of computation. It has a tape, a read/write head, and a set of rules that determine its behavior. Despite how simple it looks, a Turing Machine can simulate anything a real computer can do. This idea is the basis of the Church-Turing Thesis, which says that if something can be computed at all, a Turing Machine can do it. These sections are important because they lay out how we define what's computable and what isn't—and show that there are problems no computer will ever be able to solve, no matter how advanced technology gets.

My question this week was how has AI affected the Turing machine and will it ever replace it?

2.8 Week 8

Notes

Homework

Exercise 1: Decidability Classifications

1. $L_1 = \{M \mid M \text{ halts on itself}\}$
   RE: Yes **Decidable:** No **co-RE:** No
2. $L_2 = \{(M, w) \mid M \text{ halts on input } w\}$
   RE: Yes **Decidable:** No **co-RE:** No
3. $L_3 = \{(M, w, k) \mid M \text{ halts on } w \text{ in } \leq k \text{ steps}\}$
   Decidable: Yes

Exercise 2: True/False Decidability Questions

1. If L_1 and L_2 are decidable, then $L_1 \cup L_2$ is decidable. **True**
2. If L is decidable, then $\overline{L}$ is decidable. **True**
3. If L is decidable, then L^* is decidable. **True**
4. If L_1, L_2 are RE, then $L_1 \cup L_2$ is RE. **True**
5. If L is RE, then $\overline{L}$ is RE. **False**
6. If L is RE, then L^* is RE. **True**

Reading Summary and Questions

Sections 9.1 and 9.2 explore the idea that some problems are beyond what computers can solve, even in theory. In Section 9.1, the book introduces a language called L_d, which is not recursively enumerable. That means no Turing machine can recognize every string in this language. The authors use a technique called diagonalization (similar to a famous proof in math by Cantor) to show this. They list all possible Turing machines and their inputs, then build a language that avoids every machine on its own input. Trying to create a machine that accepts exactly this language leads to a contradiction, which proves that L_d is truly unrecognizable. Section 9.2 shifts to the universal language L_u, which contains all valid program/input pairs

where the program accepts the input. Unlike L_d, this language is recursively enumerable—a Turing machine can recognize it—but it's still undecidable. That means there's no algorithm that can always say yes or no in a finite amount of time. If the answer is "yes," the machine will eventually accept it, but if it's "no," it might run forever. These sections show that some problems can be partially solved, while others are completely out of reach.

This week I asked if some problems are only semi solvable, is there a way to identify patterns or conditions that help us guess when a solution could exist?

2.9 Week 9

Notes

Homework

Exercise 1: Order of Growth

Order the following functions from slowest to fastest:

(a) 2^{2^n}

(b) $e^{\log n} = n$

(c) $\log n$

(d) e^n

(e) $e^{2 \log n} = n^2$

(f) $\log(\log n)$

(g) 2^n

(h) n!

Solution: After simplifying and comparing growth rates, we get:

$$\log(\log n) < \log n < n < n^2 < 2^n < n! < e^n < 2^{2^n}$$

Exercise 2: Big-O Properties

Given $f, g, h : \mathbb{N} \to \mathbb{R}_{\geq 0}$:

1. $f \in \mathcal{O}(f)$.
2. $\mathcal{O}(c \cdot f) = \mathcal{O}(f)$ for $c > 0$.
3. If $f(n) \leq g(n)$ for large n, then $\mathcal{O}(f) \subseteq \mathcal{O}(g)$.
4. If $\mathcal{O}(f) \subseteq \mathcal{O}(g)$, then $\mathcal{O}(f + h) \subseteq \mathcal{O}(g + h)$.
5. If $h(n) > 0$ and $\mathcal{O}(f) \subseteq \mathcal{O}(g)$, then $\mathcal{O}(f \cdot h) \subseteq \mathcal{O}(g \cdot h)$.

Exercise 3: Polynomial and Logarithmic Growth

1. If $j \leq k$, then $\mathcal{O}(n^j) \subseteq \mathcal{O}(n^k)$.
2. $\mathcal{O}(n^j + n^k) \subseteq \mathcal{O}(n^k)$.
3. $\mathcal{O}\left(\sum_{i=0}^{k} a_i n^i\right) = \mathcal{O}(n^k)$.

4. $\mathcal{O}(\log n) \subseteq \mathcal{O}(n)$.
5. $\mathcal{O}(n \log n) \subseteq \mathcal{O}(n^2)$.

Exercise 4: Comparing Growth Rates

1. $\mathcal{O}(\sqrt{n}) \subseteq \mathcal{O}(n)$.
2. $\mathcal{O}(n^2) \subseteq \mathcal{O}(2^n)$.
3. $\mathcal{O}(\log n) \subseteq \mathcal{O}(\log^2 n)$.
4. $\mathcal{O}(2^n) \subseteq \mathcal{O}(3^n)$.
5. $\mathcal{O}(\log_2 n) = \mathcal{O}(\log_3 n)$.

Exercise 5: Sorting Algorithm Comparisons

Bubble Sort vs. Insertion Sort:

- Worst-case: $\mathcal{O}(n^2)$
- Average-case: $\mathcal{O}(n^2)$
- Best-case: $\mathcal{O}(n)$ (for insertion sort)

Insertion Sort vs. Merge Sort:

- Insertion sort: $\mathcal{O}(n^2)$ worst/avg, $\mathcal{O}(n)$ best.
- Merge sort: $\mathcal{O}(n \log n)$ in all cases.

Merge Sort vs. Quick Sort:

- Quick sort: $\mathcal{O}(n^2)$ worst, $\mathcal{O}(n \log n)$ avg/best.
- Merge sort: consistently $\mathcal{O}(n \log n)$.

In practice, quick sort tends to be faster due to better memory/cache performance, even though merge sort has a better worst-case guarantee.

Reading Summary and Questions

Sections 10.1 to 10.3 explore the idea that some problems are just too hard to solve efficiently, even with powerful computers. It starts by explaining the difference between the classes P and NP—P includes problems we can solve quickly, while NP includes problems where we can quickly check a solution, even if finding it might take forever. Most people believe P does not equal NP, but it hasn't been proven. The SAT problem is introduced as the first problem shown to be NP-complete, meaning it's as hard as any problem in NP. Cook's Theorem proves this by showing that any problem in NP can be translated into SAT. Then, the book focuses on more specific versions of SAT, like CNF-SAT and 3SAT, where formulas are written in a standardized way. Even though 3SAT seems more limited, it's still NP-complete and super useful when proving other problems are hard too. Overall, these sections explain how computer scientists figure out which problems are "hard" in a formal way, and why some problems, no matter how important, probably won't ever have a fast, general solution.

My question this week was asking if there are other ways to classify problems that dont fall into P or NP other then polynomial time reductions?

2.10 Week 10

Notes

Homework

1. **Exercise 1: Convert the following formulas to CNF**

   (a) $\varphi_1 = \neg((a \wedge b) \vee (\neg c \wedge d))$
   Using De Morgan's Laws:

   $$\neg((a \wedge b) \vee (\neg c \wedge d)) = \neg(a \wedge b) \wedge \neg(\neg c \wedge d) = (\neg a \vee \neg b) \wedge (c \vee \neg d)$$

   Final CNF: $(\neg a \vee \neg b) \wedge (c \vee \neg d)$

   (b) $\varphi_2 = \neg((p \vee q) \to (r \wedge \neg s))$
   First rewrite implication:

   $$(p \vee q) \to (r \wedge \neg s) \equiv \neg(p \vee q) \vee (r \wedge \neg s)$$

   Now negate the entire expression:

   $$\neg(\neg(p \vee q) \vee (r \wedge \neg s)) = (p \vee q) \wedge (\neg r \vee s)$$

   Final CNF: $(p \vee q) \wedge (\neg r \vee s)$

2. **Exercise 2: Determine satisfiability of the following formulas**

   (a) $\psi_1 = (a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b)$
   Try $a = \text{false}, b = \text{false}$:

   All three clauses are true.

   Conclusion: Satisfiable.

   (b) $\psi_2 = (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg(\neg p \vee r)$
   $\neg(\neg p \vee r) \equiv p \wedge \neg r$
   Combined: $(\neg p \vee q) \wedge (\neg q \vee r) \wedge p \wedge \neg r$
   Leads to contradiction on q: cannot satisfy both q and $\neg q$ simultaneously.
   Conclusion: Not satisfiable.

   (c) $\psi_3 = (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$
   Try all possible assignments of (x, y). None satisfy all clauses.
   Conclusion: Not satisfiable.

3. **Exercise 3: Encoding Sudoku constraints into Boolean formulas**

   - **Variables:** For each cell (i, j) and digit $d \in \{1, \ldots, 9\}$, define variable $X_{i,j,d}$ meaning cell (i, j) has digit d.
   - **Cell constraints:** Each cell must contain exactly one digit:

     $$X_{i,j,1} \vee X_{i,j,2} \vee \cdots \vee X_{i,j,9}$$

     and for all $d \neq d'$: $\neg X_{i,j,d} \vee \neg X_{i,j,d'}$
   - **Row constraints:** Each digit appears exactly once per row:

     $$\bigvee_{j=1}^{9} X_{i,j,d}, \quad \text{for each } i, d$$

     and for all $j \neq k$: $\neg X_{i,j,d} \vee \neg X_{i,k,d}$

- **Column constraints:** Each digit appears exactly once per column:

$$\bigvee_{i=1}^{9} X_{i,j,d}, \quad \text{for each } j, d$$

  and for all $i \neq k$: $\neg X_{i,j,d} \vee \neg X_{k,j,d}$

- **Block constraints:** Each digit appears exactly once in each 3×3 block.

  Similar to row/column constraints, applied to blocks.

Reading Summary and Questions

Sections 10.1 and 10.2 address fundamental concepts in computational complexity, focusing on how we evaluate the difficulty of solving different classes of problems. Section 10.1 introduces the complexity classes P and NP. Problems in P can be solved efficiently using a deterministic algorithm that runs in polynomial time. In contrast, problems in NP may not be easy to solve, but if a solution is provided, it can be verified efficiently. A central and unresolved question in theoretical computer science is whether P equals NP. The section also introduces the concept of NP-complete problems—these are the most difficult problems within NP. If any one of them can be solved efficiently, then all problems in NP can be as well. Section 10.2 focuses on the satisfiability problem (SAT), which asks whether there exists a truth assignment to variables in a Boolean formula that makes the formula true. SAT holds particular significance as it was the first problem proven to be NP-complete, through Cook's Theorem. It serves as a foundational reference for proving the NP-completeness of many other problems. Together, these sections establish a framework for classifying computational problems and highlight the importance of understanding problem complexity in computer science.

The question I asked this week was if there were any real life examples of NP-complete problems and how they are used in practice.

Week 11

Notes

Homework

Exercise 1: Maximal Flow & Minimal Cut

1. **Maximal Flow using Ford-Fulkerson Algorithm**

   We identify augmenting paths and increment flow step-by-step:

   - Path: $s \to a \to d \to t$: bottleneck $= 8$
   - Path: $s \to b \to d \to t$: bottleneck $= 2$
   - Path: $s \to b \to d \to c \to t$: bottleneck $= 6$

   Total flow $= 8 + 2 + 6 = 16$. Thus, the maximum flow is:

   $$\boxed{16}$$

2. **Minimal Cut**

   After running Ford-Fulkerson, we determine reachable nodes from source s in the residual graph:
   Reachable set: $S = \{s, a, b\}$ and $T = \{c, d, t\}$

   Cut edges:

- $a \to d$: capacity $= 8$
- $b \to d$: capacity $= 8$

Total cut capacity $= 8 + 8 = \boxed{16}$

3. **Is the Maximal Flow Unique?**

   No, the maximal flow is not necessarily unique. Multiple different flow assignments can achieve the same total flow.

Exercise 2: An Unknown Algorithm

1. **What value, depending on n, does the algorithm return?**

   We analyze the algorithm by counting the number of times the innermost instruction `r := r + 1` executes.

   - Outer loop: $k = 1$ to $n - 1$
   - Middle loop: $\ell = k + 1$ to n
   - Inner loop: $m = 1$ to ℓ

   The total increments of r is:

$$r = \sum_{k=1}^{n-1} \sum_{\ell=k+1}^{n} \sum_{m=1}^{\ell} 1 = \sum_{k=1}^{n-1} \sum_{\ell=k+1}^{n} \ell$$

   We simplify the expression:

$$\sum_{k=1}^{n-1} \left(\sum_{\ell=k+1}^{n} \ell \right) = \sum_{k=1}^{n-1} \left(\frac{n(n+1)}{2} - \frac{k(k+1)}{2} \right)$$

$$= \frac{n(n+1)(n-1)}{2} - \frac{1}{2} \sum_{k=1}^{n-1} k(k+1)$$

   Using the identity:

$$\sum_{k=1}^{n-1} k(k+1) = \frac{(n-1)n(n+1)}{3}$$

   We obtain:

$$r = \frac{n(n+1)(n-1)}{2} - \frac{(n-1)n(n+1)}{6} = \frac{(n-1)n(n+1)}{3}$$

   Final result:

$$\boxed{r = \frac{n(n+1)(n-1)}{3}}$$

2. **Estimate the worst-case running time in Big-O notation.**

   The innermost operation is executed $\frac{n(n+1)(n-1)}{3} \in \Theta(n^3)$, so the worst-case time complexity is:

$$\boxed{O(n^3)}$$

Reading Summary and Questions

Sections 14.1 through 14.4 cover how flow works in networks, starting with transport networks,directed graphs where each edge has a capacity, and flow moves from a source to a sink without exceeding those limits. The main goal is to figure out the maximum flow you can push through the network. Section 14.2 introduces the Max-Flow Min-Cut Theorem, which says the maximum flow equals the smallest total capacity of edges you'd need to cut to separate the source from the sink. It also explains the Ford-Fulkerson algorithm, which finds paths that can carry more flow and updates them until no more improvements can be made. Section 14.3 looks at algorithm performance, pointing out that how you pick paths affects efficiency. It introduces the Edmonds-Karp algorithm, which uses breadth-first search to reliably find max flow in polynomial time. Section 14.4 expands things to the multicommodity flow problem, where multiple flows (each with their own source and sink) have to share the same network. The tricky part is making sure the combined flow doesn't exceed any edge's capacity, which often requires using linear programming. Together, these sections lay out both the core ideas and practical tools for solving complex network flow problems.

My question this week was about the real life applications of the Max-Flow Min-Cut Theorem and Ford-Fulkerson algorithm and the benefits and drawbacks of each.

3 Synthesis

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.